

The Two Browser Signals — API 1 and API 3a

What CMED's page sends to the recorder on the same PC · AIMScribe v3 · 5 October 2026

This is the companion to "**What CMED Sends AIMS LAB**", which covers the two server-to-server messages. This document covers the two signals the **browser** sends. Every rule is taken from the code that handles the connection.

The idea in one paragraph

AIMScribe runs as a small program on **the same laptop** as the doctor's browser. CMED's page opens a connection to it over the computer's own internal network and sends two short messages: **a patient has been opened**, and **the prescription has been built**. That is all. The page never sees audio, never handles a recording, and never needs a password to do this.

1 · How the page connects

ws://127.0.0.1:5050/ws

	Detail
Address	ws://127.0.0.1:5050/ws — the same on every clinic PC. Never a name, never another machine
Protocol	A WebSocket. Messages are JSON, UTF-8
Largest message	64 KB
Password	None . See below — this surprises people
When to connect	When the doctor's page loads. Keep it open; reconnect if it drops

1.1 Why there is no password, and what replaces it

A password in a web page is not a secret — anyone can read it in the page source. So the recorder does not use one. It checks three things instead, all of which a browser sets itself and a page cannot forge:

	What is checked	What it stops
1	The exact web address your page is served from (its "origin"), against a list AIMS LAB holds	Any other website on the internet from driving the recorder
2	The address the connection was made to	A trick where a hostile site makes your own browser connect for it
3	The connection comes from this same computer	Any other machine on the network, full stop

So the one thing CMED must give AIMS LAB for this to work is the exact web address of your page — for example `https://ehr.aaloclinic.com`. Every address you will use, including staging and test sites. An address not on the list is refused with `ORIGIN_NOT_ALLOWED` and the connection never opens.

It must match exactly. `https://ehr.aaloclinic.com` and `https://www.ehr.aaloclinic.com` are two different addresses. So are `http://` and `https://`, and so is a different port number.

2 · The shape of every message and every reply

What CMED sends

```
{
  "command": "start",           <-- which signal
  "request_id": "abc-123",      <-- anything you like, up to 128 characters
  ...                          <-- the fields for that command
}
```

What comes back — always exactly one reply per message

```
{
```

```
"event":      "ack",                <-- "ack" if it worked, "error" if not
"command":    "start",              <-- the command you sent
"status":     200,                  <-- like an HTTP status
"code":       "RECORDING_STARTED", <-- ACT ON THIS
"message":    "Recording started.", <-- wording for a human; may change
"request_id": "abc-123",           <-- your id, echoed back
"data":       { ... }              <-- extra detail, per command
}
```

Act on code, never on message. The wording is for showing a person and may be reworded at any time; the code is a promise and will not change. Use `request_id` to match a reply to the message that caused it.

3 · API 1 — a patient has been opened

Send this the instant the doctor opens a patient. **The microphone starts immediately**, before any permission check finishes, so nothing of the consultation is missed.

```
{
  "command": "start",
  "request_id": "open-P0012345-1",
  "trigger": {
    "patient_id": "P0012345",
    "doctor_id": "DR0042",
    "hospital_id": "AAL0_DH0LPUR",
    "start_time": "2026-10-05T10:14:32+06:00",
    "date": "2026-10-05"
  }
}
```

Field	Required	Rule
command	yes	the word start
request_id	no	any text up to 128 characters, echoed back
trigger	yes	an object holding the five fields
trigger.patient_id	yes	letters, digits, hyphen, underscore. 1–64. No spaces
trigger.doctor_id	yes	same rule
trigger.hospital_id	yes	same rule. Must be the clinic code AIMS LAB gives you
trigger.start_time	yes	date and time with the offset , e.g. +06:00
trigger.date	yes	YYYY-MM-DD, the clinic's local date

These five values must be identical to the ones in API 2, the server-to-server message you send at the same moment. They are what ties the recording to the clinical record. Generate them once and use the same variables for both.

What comes back

```
"data": {
  "session_id": "01M3MK4J4XEDG5NDKE4H5B5XA2",    <-- KEEP THIS
  "started_at": "2026-10-05T10:14:32.865Z",
  "armed": false,
  "previous_session_stopped": true,
  "previous_session_id": "01M3MEYY...",
  "authorisation": "granted",
  "confirmation": "confirming"
}
```

Code you may receive	Meaning	What CMED should do
RECORDING_STARTED (200)	Recording, and permission confirmed	Store the session_id. Show the doctor it is recording
RECORDING_PROVISIONAL (202)	Recording. Permission is still being checked with the server	Treat exactly like success. This is normal, not a warning
SESSION_ALREADY_ACTIVE (409)	This same patient is already being recorded	Nothing. You probably sent it twice
GATE_NOT_ARMED (409)	The previous consultation has not had its prescription built yet	Ask the doctor to build the prescription for the previous patient first
AUTHORISATION_FAILED (401)	The server refused this recording, so it was stopped and deleted	Show the doctor it is not recording. Contact AIMS LAB
CLINIC_MISMATCH (401)	hospital_id is not the clinic this PC belongs to	Stop. A configuration problem — contact AIMS LAB

DOCTOR_NOT_AT_CLINIC (404)	That doctor is not registered at this clinic	Check the doctor_id
MISSING_FIELD (400)	A field is missing or malformed	Fix and resend
INVALID_IDENTIFIER (400)	An id has characters that are not allowed	Fix and resend
DEVICE_NOT_ENROLLED (423)	This PC has not been registered with AIMS LAB	Stop. Contact AIMS LAB
AGENT_NOT_READY (503)	The recorder is starting up	Wait a moment and retry

RECORDING_PROVISIONAL is the one people get wrong. It means the microphone is live and audio is being captured — the server simply has not finished confirming permission yet, which normally takes a fraction of a second. Show the doctor the same "recording" state you would for RECORDING_STARTED. Do not show a warning, and do not retry.

But a provisional start can still fail afterwards. If the server then refuses it, you receive AUTHORISATION_FAILED or DEVICE_NOT_ENROLLED **a moment later, and the recording is stopped and deleted.** So do not treat the first reply as final: keep listening, and if one of those arrives, change what the doctor sees from "recording" to "not recording". The status messages in section 6 are the reliable way to show this.

4 · API 3a — the prescription has been built

Send this the moment **Build Prescription** succeeds, at the same time as the server sends API 3b.

```
{
  "command": "prescription_built",
  "request_id": "built-P0012345-1",
  "patient_id": "P0012345",
  "session_id": "01M3MK4J4XEDG5NDKE4H5B5XA2"
}
```

Field	Required	Rule
command	yes	the words prescription_built
request_id	no	any text, echoed back
patient_id	yes	must be the patient currently being recorded
session_id	yes	the one API 1 gave you for this consultation
occurred_at	no	date and time with offset, if you want to record it

Code	Meaning	What CMED should do
GATE_ARMED (200)	Accepted	Nothing
GATE_ALREADY_ARMED (200)	You sent it twice	Nothing. Harmless
PATIENT_MISMATCH (409)	The patient named is not the one being recorded	Check you used the right session
NO_ACTIVE_SESSION (409)	Nothing is being recorded right now	The consultation already ended

5 · The most important idea in this document

Building the prescription does NOT stop the recording.

The doctor builds the prescription, prints it, hands it to the patient — and then **keeps talking**. That counselling, the part where the doctor explains how to take the medicine and what to watch for, is some of the most valuable audio in the whole study.

So prescription_built does not end anything. It **arms a gate**: it tells the recorder "this consultation has reached its end, so the next patient is allowed to close it".

The recording actually stops when the next patient is opened — when your next API 1 arrives, typically twenty to thirty seconds later.

What the doctor does	What CMED sends	What the recorder does
Opens patient A	API 1 (A)	Starts recording A
Consults	—	Recording
Presses Build Prescription	API 3a (A)	Keeps recording. Arms the gate
Prints it, hands it over, counsels	—	Still recording — this is the valuable part
Opens patient B	API 1 (B)	Closes A , starts recording B

What if the doctor finishes for the day, or goes to lunch? There is a Stop button on the recorder's own small on-screen control, which the doctor presses. **CMED does not need to send anything**, and must not try to: stopping is a clinical decision with a reason attached, and it belongs to the doctor, not to the page.

6 · Messages the recorder sends you, unprompted

As soon as the connection opens, and whenever something changes, the recorder pushes a status message. You do not have to use it — but it is how you show the doctor what is really happening.

```
{
  "event": "status",
  "state": "recording",           <-- idle | recording | paused | closing
  "is_recording": true,
  "is_paused": false,
  "session_id": "01M3MK4J...",
  "patient_ref": "P0012345",
  "armed": false,                 <-- has the prescription signal arrived?
  "authorisation": "granted",
  "confirmation": "confirmed",
  "duration_seconds": 182.4,
  "level": 0.42,                  <-- 0.0 to 1.0, how loud the room is
  "connected_clients": 1
}
```

Useful for	Which field	Suggested use
"Is it recording?"	is_recording	A red dot on the page
"Is the microphone working?"	level	A small meter. A dead microphone sits at 0.0 and is otherwise invisible until the recording is played back
"How long so far?"	duration_seconds	A timer
"Has the prescription signal landed?"	armed	A tick beside Build Prescription

7 · Commands CMED must not send

The connection also accepts stop, pause and resume. **These are not CMED's to use.** They belong to the doctor, through the recorder's own on-screen control, because each one records a clinical reason that becomes part of the permanent record — including "the patient did not consent", which erases the recording everywhere.

The only two commands CMED should ever send are start and prescription_built.

8 · Rules that are easy to miss

	Rule	Why
1	The five fields in API 1 must match API 2 exactly	They are the only link between the recording and the clinical record
2	Keep the session_id that API 1 returns	API 3a needs it. Without it you cannot arm the gate
3	Treat RECORDING_PROVISIONAL as success	The microphone is already live. It is not a warning
4	prescription_built does not stop anything	The counselling after it is valuable audio
5	Act on code, never on message	The wording may change; the code will not
6	Reconnect if the connection drops , and reconnect quietly	A dropped socket does not stop a recording in progress
7	Give AIMS LAB every web address you will use	Staging and preview addresses are refused unless listed
8	Never send stop, pause or resume	Those belong to the doctor
9	If the connection will not open at all, say "AIMScribe is not running"	It usually means the program is not started on that PC

9 · A complete working example

This is the whole integration. Everything else in this document is detail.

```
let socket, sessionId = null;

function connect() {
  socket = new WebSocket("ws://127.0.0.1:5050/ws");

  socket.onmessage = (e) => {
    const m = JSON.parse(e.data);

    if (m.event === "status") { showRecordingState(m); return; }

    if (m.command === "start") {
      if (m.code === "RECORDING_STARTED" || m.code === "RECORDING_PROVISIONAL") {
        sessionId = m.data.session_id; // keep it - API 3a needs it
        showRecording(true);
      } else {
        showProblem(m.code); // act on the code, not the message
      }
    }
  };

  socket.onclose = () => { showRecording(false); setTimeout(connect, 2000); };
  socket.onerror = () => showProblem("AIMScribe is not running on this PC");
}

// The doctor opens a patient. Build the five fields ONCE and reuse them.
function openPatient(visit) {
  socket.send(JSON.stringify({
    command: "start",
    request_id: "open-" + visit.patient_id,
    trigger: {
      patient_id: visit.patient_id,
      doctor_id: visit.doctor_id,
      hospital_id: visit.hospital_id,
      start_time: visit.start_time, // SAME object sent to API 2
      date: visit.date
    }
  }));
  sendPatientInformationFromYourServer(visit); // API 2
}

// The doctor presses Build Prescription.
function prescriptionBuilt(visit) {
  socket.send(JSON.stringify({
    command: "prescription_built",
    request_id: "built-" + visit.patient_id,
    patient_id: visit.patient_id,
    session_id: sessionId
  }));
  sendPrescriptionFromYourServer(visit); // API 3b, same start_time
}

connect();
```

10 · One page for the CMED developer

Question	Answer
Where do I connect?	ws://127.0.0.1:5050/ws — the same PC, every time
Do I need a password?	No. You need your page's web address on our allow-list instead
What must I give AIMS LAB?	The exact address of your page, including staging
How many commands?	Two. start and prescription_built
What do I keep from the reply?	session_id — API 3a needs it

Is RECORDING_PROVISIONAL bad?	No. It means recording. Treat it as success
Does prescription_built stop it?	No. It arms the gate. The next patient closes the consultation
What stops a recording then?	The next API 1, or the doctor's own Stop button
Can I send stop or pause?	No. Those belong to the doctor
What if the socket drops?	Reconnect quietly. The recording continues regardless
What if it will not connect?	Show "AIMScribe is not running". The program is probably not started
Act on which field?	code, never message
Biggest trap	The five fields must be identical in API 1 and API 2

Prepared by AIMS LAB, United International University. Companion document: "What CMED Sends AIMS LAB", covering API 2 and API 3b.